
BEAT THE BOOKIE

A PREPRINT

Group Name: GROUP K
Member: Tatsan Kantasit, Xavier Low, Mohammed Fahad Sheikh, Robbie Morris
Department of Computer Science
University College London
London, WC1E 6BT

January 3, 2025

1 Introduction

Predicting the outcomes of Premier League matches has always been an interesting challenge for sports analysts, fans, and bettors. Football is a highly dynamic sport, with multiple variables and unpredictable moments, making result prediction very difficult. However, the rise of machine learning (ML) has given this challenge a new perspective, providing tools to analyze extensive historical data, uncover hidden patterns, and make more informed predictions.

This report explores the use of ML models to predict the outcomes of English Premier League (EPL) matches, focusing specifically on games scheduled for the weekend of February 1, 2025. By analyzing historical match data, including metrics such as goals scored, shots taken, and fouls committed, we can extract meaningful insights and apply various algorithms to forecast the Full-Time Result (FTR): whether it will be a home win (H), a draw (D), or an away win (A).

The analysis incorporates standard ML methodologies, such as data preparation, feature selection, model comparison, and evaluation of prediction accuracy. By combining statistical analysis with creative problem-solving, the findings aim to provide a deeper understanding of football analytics for enthusiasts, analysts, and bettors.

2 Data Transformation & Exploration

In this section, we outline the methods used to explore, transform, and prepare the data. The process involves extracting meaningful features from raw match and player data, selecting the most relevant features for our prediction model, and preprocessing the data to ensure compatibility with the machine learning algorithms we plan to use.

2.1 Data Collection and Cleaning

The initial dataset was collected by scraping historical EPL match and player statistics from reliable sources, particularly FBRef, using Selenium, a Python automation library. The dataset includes a wide range of detailed metrics. For players, it captures parameters such as pass completion percentage (Cmp%), shot-creating actions (SCA), tackles (Tk1), goals plus assists (G+A), and total shots (Sh), among others. For matches, it includes metrics like expected goals (xG), goal-creating actions (GCA), errors leading to shots (Errr), passes into the final third (1/3), and penalty attempts (PKatt), among others. For a complete list of parameters and further details, please refer to the project notebook.

The dataset covers all EPL seasons from 2017-18 to 2024-25 (current), allowing us to account for the historical performance of players and teams. The reason for limiting the data to 2017-18 onward is the unavailability of player data before that period.

Unlike most research papers that rely on simplified player data derived from sources such as FIFA ratings, our dataset contains detailed player statistics for each match (Raju et al.). This detailed approach enables more accurate and relevant analysis of player performance. Additionally, we extracted over 80 parameters per match, significantly exceeding the feature sets typically used in existing research papers, which often rely on simplified data. This comprehensive data collection provides a richer foundation for predictive modeling [2].

The raw data was cleaned by removing duplicate rows and columns, columns consisting entirely of null values, and unhelpful columns like the 'Competition' column (which contained a constant value throughout).

2.2 Feature Extraction

Feature extraction involves selecting and deriving relevant attributes from the raw data to be used as inputs for machine learning models. Created features fall into three main categories: standard features, Exponential Moving Averages (EMA), and advanced ratings (Elo and Pi-Rating).

It is worth noting that the player statistics dataset was only utilized for calculating the Elo and Pi-Rating of the teams, as individual player-level attributes are less relevant in the context of team-level match outcomes. This approach simplifies the model while focusing on metrics that directly influence match results.

2.2.1 Standard and Composite Feature Creation

Several standard features were extracted based on historical match data to capture key aspects of performance that influence match outcomes. Although these features provide a very detailed view of performance, the sheer number of metrics can overwhelm machine learning algorithms, leading to potential overfitting and computational inefficiency. To address this, metrics were grouped into composite metrics, representing specific aspects of football, such as attacking, defensive, and possession capabilities.

Grouping the metrics into categories allows the machine learning model to focus on higher-level patterns rather than minor variations in individual metrics. For example, combining tackles won, interceptions, and blocks into a Defensive Capability Metric (DCM) cover the overall defensive performance, which is more impactful for predictions than considering each feature separately. The full list of grouped metrics is shown in Table 1. These groupings not only simplify the dataset but also retain the understandability and relevance of the features.

Metric Name	Description	Importance
Win Streak	The number of consecutive matches won by a team prior to the current match.	Provides insight into team momentum.
Points	Points earned based on match results (3 for a win, 1 for a draw, 0 for a loss).	Determines league rankings and team success in particular season.
Recent Form	A rolling of match results (e.g., last 5 matches).	Captures short-term performance trends and team consistency.
Defensive Capability Metric (DCM)	Combines features like tackles won, interceptions, blocks, and recoveries.	Provides a comprehensive evaluation of a team's defensive strength.
Attacking Capability Metric (ACM)	Evaluates offensive contributions, including goal-scoring, chance creation, and passing in attacking areas.	Highlights a team's ability to generate and capitalize on scoring opportunities.
Possession Influence Metric (PIM)	Measures a team's ability to control the game and influence possession effectively.	Crucial for understanding dominance in ball control and strategic gameplay.
Efficiency Metric (EM)	Assesses overall efficiency in converting chances and performing successful defensive actions.	Balances offensive and defensive effectiveness to gauge productivity.
Expected Performance Metric (EPM)	Combines expected goals, assists, and defensive contributions.	Helps analyze over or under-performance relative to expected values.
Physical Intensity Metric (PIM)	Combines tackles, interceptions, recoveries, and total distance covered.	Reflects the physical effort and intensity of a team's or performance.
Creativity Metric	Measures chance creation, progressive passes, and final third involvement.	Highlights a team's ability to break down opposition defenses.

Table 1: Standard and Composite Metrics for Football Match Analysis

2.2.2 Exponential Moving Average Feature Creation

As most match statistics (number of possessions, number of passes etc.) are not known prior to a match being played, directly using them as features would result in an inability to predict future matches (Henrik Thustrup). We can instead use EMA features – that is, for any particular match, take the weighted average of the match statistics from span previous matches played by that team. EMAs are calculated as follows:

$$EMA_t = \alpha \cdot x_t + (1 - \alpha) \cdot EMA_{t-1}, \quad \text{where} \quad \alpha = \frac{2}{\text{span} + 1}$$

The decay factor, α , results in greater importance being given to recent matches. The optimal α was found via cross-validation.

2.2.3 Elo System

Elo Rating is a widely used system for measuring the relative skill levels of players or teams (Hvattum and Arntzen). In our methodology, the Elo rating for a player and team starts at an initial value of 1500 and is updated after each match based on the match outcome, the strength of the opponent, and position-specific performance metrics. The formula for updating Elo ratings is:

$$R' = R + K \cdot (\text{Result} - \text{Expected Result}),$$

Where R is the current Elo rating, R' is the updated Elo rating, and K is the dynamic rating adjustment factor, typically set to 32. `Result` represents the outcome of the match (1.0 for a win, 0.5 for a draw, and 0.0 for a loss), and `Expected Result` is the probability of winning, calculated based on the difference in Elo ratings between teams or players.

Our system also integrates performance metrics that match to player positions to dynamically adjust their Elo ratings. For example, a higher weight is given to a goal compared to tackle when we are considering a striker. Player ratings are aggregated at the team level using position-specific weights to calculate an overall team Elo rating. In our implementation, we have given forwards a slightly higher rating than midfielders.

2.2.4 PI Rating System Overview

The PI Rating System dynamically evaluates team performance by adjusting ratings based on match outcomes and contextual factors like goal differences in a match, team strength and particularly distinguishing between home and away performances. It calculates the `Expected Goal Difference` for home and away matches separately, considering team ratings and location. The `Observed Error in Goal Difference`, the difference between expected and actual results, is tracked to measure deviations, while a `Weighted Error` applies logarithmic scaling to focus on the significant deviations. Ratings are then updated using these weighted errors, accounting for direct performance and the opponent's strength through predefined weight factors [6]. This approach ensures adjustments that reflect real world complexities of home advantage and team dynamics.

Unlike the simpler Elo system, which applies uniform win/loss-based adjustments and does not distinguish between home and away performance, the PI Rating System offers a more advanced evaluation. Its use of dynamic and context-sensitive updates provides more detailed control over ratings, allowing for greater accuracy and adaptability to varying match scenarios [6].

2.3 Feature Selection

Feature selection is the process of identifying the most relevant features for training machine learning models. This step helps reduce the complexity of the model, improve performance, and prevent overfitting.

2.3.1 Feature Correlation Analysis

Some insight into which features might be the most useful can be drawn from correlating our features with home match wins. If a feature is highly correlated to winning, it is likely to contribute more to our machine learning model's predictive capacity. As shown in Figure 1, the PI and Elo rating system and a variety of EMA features are the most correlated with winning, and we see similar inverted EMA features for the most inversely correlated with winning.

2.3.2 Remove Highly Correlated Features

Then a comparison between the feature importance scores from a Random Forest model was performed. Random Forest evaluates feature importance based on how much each feature contributes to reducing impurity across its decision trees. By ranking features using this metric, we identified the most significant subset.

As seen in Figure 2, some pairs of features in this subset have a very high pairwise correlation (> 0.9). For each of these pairs, the less important feature was selected for removal. When features are highly correlated (e.g. the Defensive Capability Metric (DCM) and its components like number of tackles/interceptions), the model may struggle to differentiate their individual contributions, leading to redundancy and inflated variance in coefficients. Cutting highly correlated features therefore reduces model complexity, prevents overfitting, and enhances numerical stability. Additionally, it decreases training time and allows the model to focus on non-redundant signals.

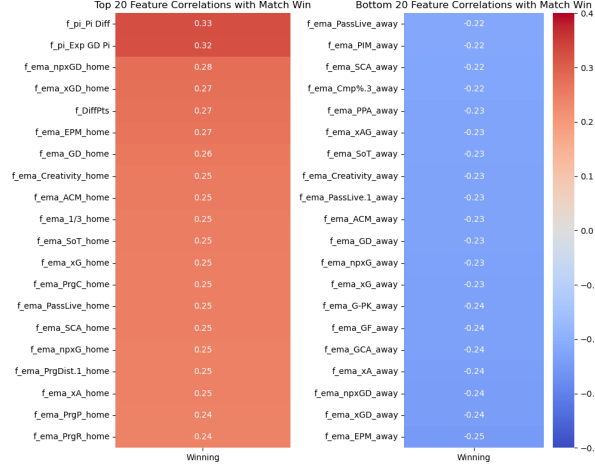


Figure 1: Top and Bottom 20 Features Correlations with a Home Win.

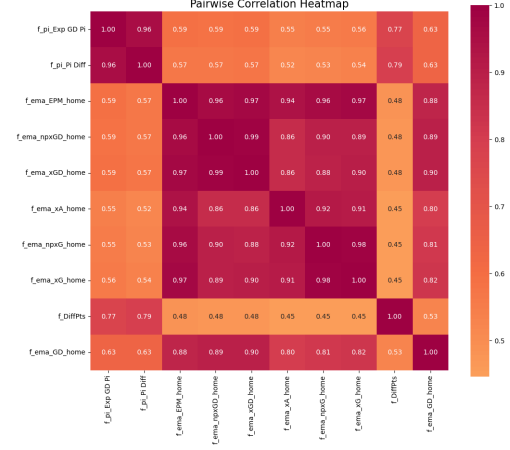


Figure 2: Pairwise Correlation between Top 10 Features.

2.4 Data Preprocessing

The data preprocessing stage is very important for preparing the dataset for machine learning algorithms. Categorical variables, that are not numerical such as match results, were encoded using Label Encoding, assigning unique integer values to each category. Missing values for each feature were imputed by computing the mean. Additionally, features were normalized using `StandardScaler` to standardize their distribution with a mean (μ) of 0 and a standard deviation (σ) of 1, ensuring the data aligns with the input expectations of many ML models.

2.5 Data splitting

The data set was divided into train and test sets to ensure that the models were evaluated on unseen data. The time-sensitive nature of our data presents a unique challenge: when predicting results, models should only have been trained on matches that occurred previously. For example, it would be illogical to train on 2024 data and predict 2017 matches. We therefore used an 80-20 train-test split with shuffling disabled to avoid any risk of data leakage. The train set is further split into train and validation folds during model cross-validation. Note that the train-test split is for the purposes of evaluating the models' performance. The final model used to predict the **given test set** is trained on 100% of the data.

2.6 Final Dataset Overview

Our final data set has 2,591 rows and 159 features, with a class distribution of 1,153 home wins, 839 away wins, and 599 draws. It follows that our training data set has 2072 rows with a class distribution of 925 home wins, 673 away wins, and 474 draws, and our testing data set has 519 rows with a class distribution of 228 home wins, 166 away wins, and 125 draws. Notice that there is a significant class imbalance, which we will address later.

3 Methodology Overview

3.1 Related Works & Background Reading

Machine learning (ML) techniques have been widely used to predict football match outcomes, particularly in the context of the English Premier League (EPL). Early statistical models, such as those by Maher [11] and Dixon and Coles [7], used Poisson distributions to predict goals and outcomes. As for recent studies they have utilized various ML algorithms. For instance, Kundu et al. (2020) applied gradient boosting models which achieved very high accuracy [10]. Similarly, Malamatinos et al. (2022) used k-NN, LogitBoost, support vector machines (SVMs), and CatBoost, with CatBoost outperforming others model for the Greek Super League [12]. Ensemble models combining multiple techniques have also shown promise, such as the work by Stübinger et al. (2020) which leveraged player metrics [14].

While models like SVM, Random Forest, and XGBoost are very popular in majority of our background reading, recent research has explored creative approaches to increase the prediction accuracy. For example, Beal et al. combined

sentiment analysis from Guardian articles with traditional ML models, demonstrating improved performance [2]. Additionally, injury analysis has been integrated into ML models, as shown by Carloni et al. (2021) and Hucaljuk and Rakipović (2011), who used player fitness data to refine predictions (Chang et al.). These approaches highlight the potential for incorporating contextual and real-time data sources to enhance model reliability.

Therefore, in this report, we will focus on the established models like SVM, Random Forest, and XGBoost due to their prevalence in the literature. However, we aim to diverge from previous research by experimenting with GRUs (Gated Recurrent Units), neural networks, and Elastic Net regression. We also aim to combine all or some of these experimental models into one meta-model that (hopefully) has superior performance.

3.2 Model Development and Integration

This section is structured as a pipeline. The approach begins with a baseline model for benchmarking, then adds in more complex and creative models, and ends with the combination of the individual models into one meta-model. This final model, in theory, should give us optimal predictive accuracy by leveraging the unique strengths of each model.

3.2.1 Baseline Model Setup: Linear Regression

Our baseline model is a linear regression on the Expected Goals (xG) paired with manual classification. We train two separate linear regression models: one for predicting the expected goals for the home team (home_xG) and another for the away team (away_xG). Once the expected goals are predicted for both the home and away teams, we manually classify the outcome of the match by comparing the number of goals. This approach ensures interpretability and simplicity, making it easy to analyze the decision-making process of the model. By comparing this model's performance with more complex models, we can tell the value added by additional layers of complexity in our predictive pipeline.

3.2.2 Support Vector Machines (SVM)

Support Vector Machines (SVMs) are highly effective supervised learning models, particularly useful when dealing with high-dimensional feature spaces. SVMs aim to find the optimal hyperplane that separates data points belonging to different classes (e.g., win, loss, draw) by maximizing the margin between the closest data points (support vectors).

The dimensionality of the input features was reduced by considering Random Forest importance scores. This ensures faster training and reduces overfitting. For this study, we used an SVM model and optimized its hyperparameters using grid search cross-validation. This combined approach improved interpretability, reduces computational overhead, and delivers competitive accuracy. [16]

3.2.3 Ensemble Learning with Random Forest and XGBoost

Random Forest is an ensemble learning method that builds multiple decision trees during training and combines their outputs to produce predictions. Each decision tree is trained on a random subset of the data and features, reducing the risk of overfitting. In this case, Random Forest was used for classification (predicting match outcomes). The model then aggregates the predictions of all trees. Random Forest was employed to capture nonlinear relationships and evaluate feature importance, using its advantage of its strength against noise and overfitting.

XGBoost (Extreme Gradient Boosting) is an advanced ensemble method based on gradient-boosted decision trees. It is known for its speed making it suitable for large datasets and complex problems. XGBoost works by iteratively adding decision trees to minimize a loss function. Each new tree corrects errors made by the previous trees. The model incorporates regularization techniques to prevent overfitting, making it more robust compared to traditional boosting methods. As XGBoost and RandomForest are pre-trained models, optimization was performed via tuning hyperparameters.

3.2.4 ElasticNet Regression

ElasticNet is a linear regression model that combines L1 (Lasso) and L2 (Ridge) regularization techniques. This combination allows ElasticNet to perform well when the dataset has a large number of correlated features, which is often the case in football match prediction. ElasticNet works by minimizing the loss function with a penalty term that balances both types of regularization. The loss function is defined as:

$$L(\theta) = \mathcal{L}(y, \hat{y}) + \lambda_1 \sum_{j=1}^p |\theta_j| + \lambda_2 \sum_{j=1}^p \theta_j^2$$

where $\mathcal{L}(y, \hat{y})$ is the primary loss function (e.g., cross-entropy loss or mean squared error), θ_j represents the model parameters (weights), λ_1 is the L1 regularization coefficient, which promotes sparsity, and λ_2 is the L2 regularization coefficient, which penalizes large weights. We use L1 and L2 regularization from ElasticNet in combination with a Multi-Layer Perceptron (MLP) to get our match result predictions. Referencing this paper, we assigned the L1 and L2 values of 0.002 and 0.0005. ElasticNet helps reduce over-fitting and select important features automatically, making it a robust choice for predicting continuous outcomes like the number of goals (Anfilets et al.).

3.2.5 Neural Network Integration (GRU + MLP)

To model sequential dependencies and temporal patterns in football match data, Gated Recurrent Units (GRUs) were also explored and integrated. GRUs processed time-series features such as recent form and EMA metrics to predict trends in match outcomes. GRUs are particularly effective for situations where recent performance has a large influence on future outcomes.

To complement GRUs, a Multi-Layer Perceptron (MLP) was added as a secondary stage. The MLP took outputs from the GRU and combined them with static features (e.g., team Elo ratings) to capture non-sequential relationships. This hybrid model improved generalization by merging temporal and static features (Yeung et al.).

3.2.6 Model Stacking

Model stacking is an ensemble learning technique used to improve predictive performance by combining predictions from multiple base models (SVM, Random Forest, XGBoost, and GRU+MLP in our approach) via a meta-model, a Logistic Regression classifier. In order to fit the meta-model, the base models' predictions on the train set (also known as **meta-features**) are required. Cross-validation is used to obtain these meta-features without overfitting.

The meta-model is then fitted on the meta-features to learn the optimal combination of the base models' outputs. The base models are then retrained on the entire training dataset, and their predictions on the previously unseen test set are obtained. Finally, these meta-features (corresponding to the test set) are passed to the meta-model to obtain its prediction on the test set. See the full model architecture in Figure 3 & 4.

This approach ensures that interactions between the base models' outputs are effectively captured and utilized to produce more accurate and robust predictions. The use of a Logistic Regression meta-model also provides a balance between interpretability and performance.

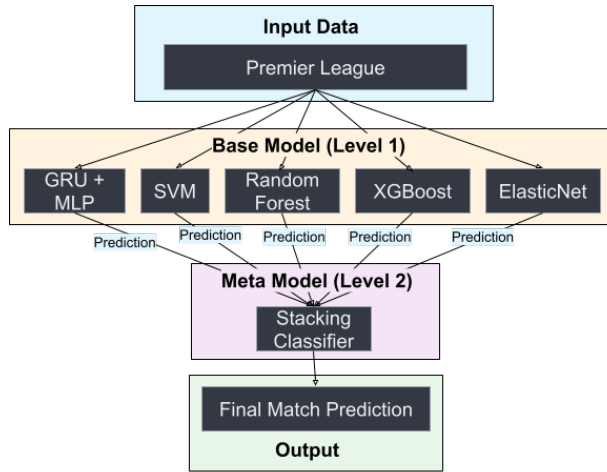


Figure 3: Stacking Model Architecture

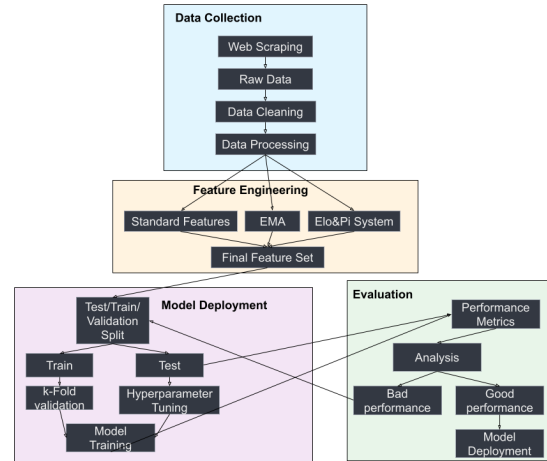


Figure 4: Model Deployment Process

4 Model Training & Validation

4.1 Model Training Process

The training process was implemented using **PyTorch**, following the systematic iterative approach laid out in Figure 4 across multiple model architectures. The process began with data preparation, where features were normalized and split into training and validation sets using time series cross-validation to maintain temporal ordering. Each model required

different training configurations: the GRU+MLP hybrid was trained with a learning rate of 0.0003 for 100 epochs, while the standalone MLP and ElasticNet MLP were trained for 50 epochs with a learning rate of 0.001, both utilizing the Adam optimizer for parameter updates. LeakyReLU activation functions were employed in the ElasticNet MLP architecture to handle negative values and prevent dying ReLU problems, while the hybrid GRU+MLP utilized standard ReLU activations.

The training loop was structured consistently across models, with each epoch consisting of forward passes to compute predictions and losses, followed by backpropagation for gradient calculation and parameter updates. Model performance was continuously monitored through validation metrics, with model checkpoints saved when improvements were observed. Different regularization techniques were employed based on the model architecture: dropout layers (0.4, 0.3, 0.1) for neural networks, L_1 ($\lambda = 0.002$) and L_2 ($\lambda = 0.0005$) regularization for ElasticNet, and tree pruning for Random Forest with 100 estimators. Learning rate scheduling was implemented to reduce rates by a factor of 0.1 when validation metrics plateaued, while early stopping with a patience of 5 epochs prevented overfitting. The training process maintained proper gradient handling through PyTorch’s autograd system, including gradient zeroing at each iteration and gradient clipping to prevent explosion. For tree-based models, BayesSearchCV (Random Forest and XGBoost) and GridSearchCV (SVM) with 10-fold time series cross-validation were used to optimize hyperparameters, ensuring robust model performance while respecting the temporal nature of the data.

4.2 Final Stacking Model Training Process

The stacking model is trained in two stages: base model training and meta-model training. Each base model is trained on a subset of the training data (folds) and predicts probabilities for the held-out fold, creating out-of-fold predictions. The predictions by each of the base models are stacked to form the train meta-features, which are then used to train the meta-model (Logistic Regression). Subsequently, the base models are retrained on the full train set. Their predictions on the test set are then stacked to create test meta-features. Finally, the meta-model uses these test meta-features to predict the final output. This hierarchical approach captures diverse patterns from individual models while leveraging the meta-model to improve predictive performance.

4.3 Validation

Validation plays a key role in ensuring the models perform well on unseen data by testing their accuracy on a separate validation set. This process helps identify and address issues like overfitting or underfitting.

Because we have a time series dataset, in most cases it does not make sense to use normal cross-validation. Instead, we implemented k-folds cross-validation as our primary validation strategy. This method is specifically made for time-based data, helping to mitigate bias and provide accurate performance estimates while respecting the temporal structure of the dataset. The data set is divided into k subsets (folds), typically $k = 10$. In each iteration, the model is trained on the first i folds and validated on the $i + 1$ -th fold. This process is repeated $k - 1$ times, starting from $i = 1$ to $i = k - 1$, with the size of the training set growing incrementally as the next fold is added. This approach avoids data leakage, and ensures that the evaluation respects the temporal order of the data. The final evaluation metrics are averaged across all iterations to provide a comprehensive assessment of the model’s performance.

Evaluation metrics such as accuracy, precision, recall, and F1-score assess model effectiveness. **Accuracy** indicates the proportion of correctly classified instances but may be unreliable for imbalanced datasets. **Precision** measures true positives among predicted positives, focusing on minimizing false positives. **Recall** assesses true positives among actual positives, emphasizing false negative reduction. The F1-score combines precision and recall into a single metric, offering a balanced measure of a model’s performance, especially useful for imbalanced datasets. These metrics provide insights for model selection and optimization.

4.4 Hyperparameter Tuning

The pre-trained models (XGBoost and Random Forest) were tuned via **Bayesian Optimization** over their hyperparameters. This approach entails defining a search space for the hyperparameters of interest, and evaluating the model’s performance with different combinations of hyperparameters. After a specified number of iterations, the hyperparameter combination that yields the highest score is saved for future use.

4.5 Oversampling

Class imbalance is a significant issue in football match prediction, as **home wins** are more frequent than **draws** or **away wins**. For our dataset, there is a 2:1:1 ratio of home wins to draws and away wins, indicating moderate imbalance.

To address this, we attempted to employ oversampling to ensure the minority classes are sufficiently represented. In particular, we tried resampling the dataset with **Random Oversampling** and **Synthetic Minority Oversampling Technique (SMOTE)**. Both oversampling methods aim to generate *synthetic samples* for the minority class to reduce model bias; Random oversampling duplicates real samples, while SMOTE interpolates between existing samples.

However, when we tried training the models on the resampled data, it was quickly apparent that although oversampling increases prediction precision for the minority class (Draws), it leads to a **decrease in both F1-scores and accuracy** across all models. Therefore, we ultimately decided to not oversample the data.

5 Results

This section presents the outcomes obtained using various machine learning models on the test set. Performance metrics, such as **accuracy**, **precision**, **recall**, and **F1 score**, are used to evaluate the effectiveness of each approach. The results are compared to identify the best-performing model and analyze key findings.

5.1 Model Performance Comparison

Model	Accuracy	Precision	Recall	F1 Score
Linear Regression	48.84%	37.72%	48.84%	42.38%
SVM	51.74%	48.60%	51.74%	45.30%
Random Forest	54.05%	41.04%	54.05%	46.09%
XGB	53.28%	40.33%	53.28%	45.88%
Elastic Net	52.90%	40.09%	52.90%	44.88%
MLP	52.90%	39.83%	52.90%	45.21%
GRU	53.09%	40.14%	53.09%	45.56%
GRU+MLP	53.09%	52.21%	53.09%	46.26%
Stacking Ensemble	54.25%	41.00%	54.25%	46.59%

Table 2: Model Performance Comparison

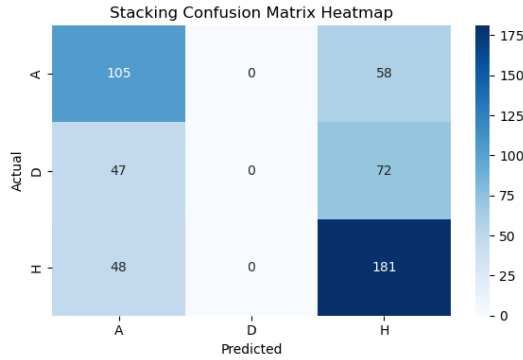


Figure 5: Confusion Matrix for Stacking Model

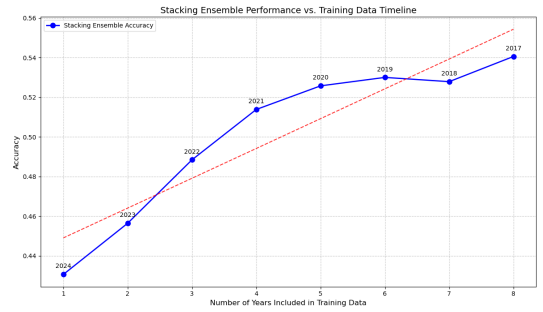


Figure 6: Model Accuracy against year included.

5.2 Key Insights

Among the individual models, the RandomForest model achieved the highest accuracy of 54.05% and an F1-score of 46.09%, showcasing solid performance. However, our final model, the stacking ensemble, outperformed all individual models, achieving the highest accuracy of 54.25% and an F1-score of 46.59%. While the stacking model surpassed the baseline models such as Linear Regression and SVM by a notable margin, its improvement over models like XGBoost and RandomForest was smaller. This highlights the diminishing returns in combining models at the upper end of performance, although the ensemble approach effectively utilizes the complementary strengths of individual models to achieve the best overall metrics.

The lack of performance difference between the GRU (53.09%) and GRU+MLP (53.09%) could be due to overfitting from added complexity in the MLP, feature interactions between GRU versus MLP (patterns captured by GRU might be

more relevant than the non-linear transformations from MLP), or higher optimization difficulty due to the more complex architecture, potentially getting stuck in suboptimal local minima. To verify if there is a significant difference in the performance of the two models, validation would have to be performed multiple times with different random seeds.

The confusion matrix for the stacking ensemble (Figure 5) provides additional insights into its performance. While the model demonstrated strong precision and recall for predicting Home and Away wins, it failed to classify any Draw outcomes, which adversely affected its F1-score. This consistent difficulty in predicting draws is not unique to the stacking ensemble but is observed across all models, as none were able to effectively capture the nuances of evenly matched teams or defensive gameplay strategies. This suggests that all models may be overfitting to the more frequent outcomes of wins and losses, while failing to differentiate features that signify draws. Furthermore, this limitation underscores the inherent unpredictability of football matches, where draws typically account for 20%-30% of games. Developing features specifically built to capture these draws may help address this inconsistency.

Feature importance analysis consistently highlighted key predictors of match outcomes. Metrics such as Pi Difference, Expected Goal Difference (fpiExp , GD), and EMA metrics for home teams were positively correlated with winning. Conversely, higher values in away team EMA metrics exhibited a strong negative correlation with the likelihood of victory, emphasizing their inverse relationship with match outcomes. These insights reaffirm the importance of strategically leveraging these features during model development, when looking back at Figure 2. We also see from looking at Figure 6 that all the years of data are positive contributors to the current predictions.

In conclusion, while the stacking ensemble delivered the best overall performance, its inability to accurately predict draws remains a notable limitation. Future work could focus on refining feature engineering, addressing class imbalance, and incorporating additional data to improve the model’s ability to capture these less common but significant outcomes.

6 Final Predictions on Test Set

Below in Table 3 is the match prediction results for the 1st February 2025 using our final stacking model.

Home Team	Away Team	Full Time Result
Arsenal	Manchester City	Home
Brentford	Tottenham	Home
Bournemouth	Liverpool	Away
Nott’ham Forest	Brighton	Home
Chelsea	West Ham	Home
Newcastle Utd	Fulham	Home
Ipswich Town	Southampton	Home
Manchester Utd	Crystal Palace	Home
Everton	Leicester City	Home
Wolves	Aston Villa	Away

Table 3: Match Prediction for 01/02/2025

7 Conclusion

In this report, we explored various machine learning models and methodologies to predict Premier League football match outcomes. Our approach included testing models such as GRU, MLP, Random Forest, ElasticNet, and XGBoost, as well as advanced techniques like stacking ensembles. While the stacking ensemble delivered the highest accuracy, it outperformed one of the individual models by only a small margin, emphasizing the need to weigh complexity against incremental gains. Notably, we introduced features like Pi-Rating, a novel concept in EPL prediction, which emerged as one of the most significant features in terms of importance. This demonstrates the potential of incorporating innovative methodologies to enhance predictive accuracy and provide fresh insights into football match prediction.

Predicting football matches is challenging due to the unpredictable nature of the game. By using machine learning we tackle this complexity through extracting meaningful patterns from historical data. However, as seen in our report, success in this prediction requires careful feature engineering, robust model evaluation, and a combination of techniques customized to the problem. Looking ahead, the integration of additional data sources, such as injuries, workload, or sentiment analysis from media and fan commentary, could further enhance the predictive capabilities of machine learning models. By continuing to refine these methodologies, machine learning has the potential to not only predict outcomes more accurately but also provide valuable insights for teams, analysts, and fans, making it an exciting area of research.

References

- [1] Rahul Baboota and Harleen Kaur. Predictive analysis and modelling football results using machine learning approach for English Premier League. *International Journal of Forecasting*, vol. 35, no. 2, Apr. 2019, pp. 741–755. <https://doi.org/10.1016/j.ijforecast.2018.01.003>.
- [2] Ryan Beal, et al. Combining machine learning and human experts to predict match outcomes in football: A baseline model. *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 17, 18 May 2021, pp. 15447–15451. <https://doi.org/10.1609/aaai.v35i17.17815>.
- [3] Daniel Berrar, et al. A data- and knowledge-driven framework for developing machine learning models to predict soccer match outcomes. *Machine Learning*, 24 Sept. 2024. <https://doi.org/10.1007/s10994-024-06625-9>. Accessed 2 Oct. 2024.
- [4] Victor Chang, et al. Football analytics: Assessing the correlation between workload, injury and performance of football players in the English Premier League. *Applied Sciences*, vol. 14, no. 16, 16 Aug. 2024, pp. 7217–7217. <https://doi.org/10.3390/app14167217>.
- [5] Belinda Choi, et al. Predicting football match outcomes with machine learning approaches. *Mendel*, vol. 29, no. 2, 20 Dec. 2023, pp. 229–236. <https://doi.org/10.13164/mendel.2023.2.229>.
- [6] Anthony Costa Constantinou and Norman Elliott Fenton. Determining the level of ability of football teams by dynamic ratings based on the relative discrepancies in scores between adversaries. *Journal of Quantitative Analysis in Sports*, vol. 9, no. 1, 30 Mar. 2013, pp. 37–50. <https://doi.org/10.1515/jqas-2012-0036>. Accessed 25 July 2020.
- [7] Mark J. Dixon and Stuart G. Coles. Modelling association football scores and inefficiencies in the football betting market. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, vol. 46, no. 2, May 1997, pp. 265–280. <https://doi.org/10.1111/1467-9876.00065>. Accessed 17 Dec. 2024.
- [8] Henrik Thustrup. Assessing football team performance using exponential moving average (EMA). *Medium*, 11 Aug. 2023. <https://medium.com/@HenrikThustrup/assessing-football-team-performance-using-exponential-moving-average-ema-a6e8ecce5b04>. Accessed 17 Dec. 2024.
- [9] Lars Magnus Hvattum and Halvard Arntzen. Using ELO ratings for match result prediction in association football. *International Journal of Forecasting*, vol. 26, no. 3, 2010, pp. 460–470. https://econpapers.repec.org/article/eeeintfor/v_3a26_3ay_3a_3ai_3a3_3ap_3a460-470.htm. Accessed 5 Jan. 2024.
- [10] Tuhin Kundu, et al. Predicting English Premier League matches using classification and regression. *Algorithms for Intelligent Systems*, 28 Aug. 2020, pp. 555–568. https://doi.org/10.1007/978-981-15-5077-5_50. Accessed 15 Apr. 2023.
- [11] M. J. Maher. Modelling association football scores. *Statistica Neerlandica*, vol. 36, no. 3, Sept. 1982, pp. 109–118. <https://doi.org/10.1111/j.1467-9574.1982.tb00782.x>.
- [12] Marios-Christos Malamatinos, et al. On predicting soccer outcomes in the Greek league using machine learning. *Computers*, vol. 11, no. 9, 31 Aug. 2022, p. 133. <https://doi.org/10.3390/computers11090133>. Accessed 29 Jan. 2023.
- [13] M. A. Raju, et al. Predicting the outcome of English Premier League matches using machine learning. *IEEE Xplore*, 2020. <https://ieeexplore.ieee.org/document/9350327>. Accessed 17 Dec. 2024.
- [14] Johannes Stübinger, et al. Machine learning in football betting: Prediction of match results based on player characteristics. *Applied Sciences*, vol. 10, no. 1, 19 Dec. 2019, p. 46. <https://doi.org/10.3390/app10010046>. Accessed 14 Jan. 2020.
- [15] Yeung, Calvin, et al. “Evaluating Soccer Match Prediction Models: A Deep Learning Approach and Feature Optimization for Gradient-Boosted Trees.” *ArXiv.org*, 2023. arxiv.org/abs/2309.14807. Accessed 17 Dec. 2024.
- [16] Dewi, Christine. “Random Forest and Support Vector Machine on Features Selection for Regression Analysis.” *International Journal of Innovative Computing, Information ResearchGate*, 1 Dec. 2019. www.researchgate.net/publication/341508483_Random_Forest_and_Support_Vector_Machine_on_Features_Selection_for_Regression_Analysis. Accessed 17 Dec. 2024.
- [17] Anfilets, Sergei, et al. “DEEP MULTILAYER NEURAL NETWORK for PREDICTING the WINNER of FOOTBALL MATCHES.” *International Journal of Computing*, 31 Mar. 2020, pp. 70–77. <https://doi.org/10.47839/ijc.19.1.1695>.